

HealthEasy-G — Hospital Management System

Group: Architects

Project: HealthEasy

CSCD602 Advance Software Engineering — Capstone Project

Capstone Project Coursework — Group Capstone Project, Individual Submission

University of Ghana, Department of Computer Science

Student:

Ernest Nketia Asubonternng (Lead)

Student ID: 22424715

Software architecture, Ollama clinical assistant and GSTG rule engine, Next.js App Router, PostgreSQL and Prisma, HeFRA/GHS compliance, multi-facility management

An enterprise Hospital Management System for Ghana's healthcare ecosystem, built with Next.js 16 (App Router), TypeScript, Tailwind CSS, PostgreSQL and Prisma ORM, and aligned to **HeFRA, GHS, NHIA G-DRG, DHIMS2** and **Data Protection Commission** standards.

Live application: <https://healtheasy-g.vercel.app>

Examiner credentials are not published here. They are supplied in Links.txt with the Sakai submission, in line with the coursework rule against placing passwords in publicly accessible repositories.

1. Project Overview

Ghanaian facilities routinely run on paper folders and disconnected spreadsheets. A patient seen three times can accumulate three folders, three NHIS numbers and three conflicting allergy lists. HealthEasy-G replaces that with a single record that follows the patient from the registration desk, through triage, consultation, the laboratory, radiology, the pharmacy and the cashier, to the NHIS claim — without anyone retyping a name.

2. Main System Features

Area	Capability
Patient administration	Master Patient Index, Ghana Card and NHIS validation, duplicate prevention with merge, queue routing
Clinical care	Triage vitals with ESI scoring and server-derived abnormality alerts, OPD/emergency consultation, ICD-10 coding, clinical orders
Diagnostics	Laboratory orders with specimen barcoding, result entry and separate verification; PACS radiology requests and radiologist sign-off

Inpatient	Ward bed board, admission/discharge/transfer, medication administration chart
Pharmacy	FEFO batch stock, prescription verification, transactional dispensing with stock deduction
Revenue	Cashier collections and receipts, server-recomputed invoice totals, refund authorisation control
Insurance	NHIS G-DRG claim lines assembled from clinical data, claim batching for CLAIM-it submission
Governance	Live DHIMS2 monthly return, HeFRA licence tracking, immutable audit trail, 19-role RBAC
Clinical AI	Ollama-backed prescribing assistant with a Ghana Standard Treatment Guidelines rule-engine fallback

3. Technology Stack

Layer	Technology
Framework	Next.js 16 (App Router, React Server Components)
Language	TypeScript 5 (strict)
UI	React 19, Tailwind CSS 4, Radix UI, Tabler Icons
Data	PostgreSQL 16 with Prisma ORM 5
Authentication	HMAC-SHA256 signed session cookies (Web Crypto)
Passwords	bcrypt (10 rounds)
Clinical AI	Ollama JS SDK with a deterministic GSTG rule-engine fallback
Testing	Node.js built-in test runner with tsx
CI	GitHub Actions (typecheck, test, build against a live PostgreSQL service)
Hosting	Vercel; Neon serverless PostgreSQL

4. Accessing the Application

1. Open <https://healtheasy-g.vercel.app>.
2. The landing page is public. Select **Staff sign in**.
3. Sign in with the credentials supplied in Links.txt.

There is no separate admin URL — the administrator signs in at the same portal and the interface adapts to the role held by the account.

Test and administrator credentials: see Links.txt in the Sakai submission.

5. Running Locally

Prerequisites: Node.js 20+, PostgreSQL (local instance, a cloud instance, or both).

```
git clone https://github.com/NketiaAsubontengErnest/HealthEasy-G.git
```

```
cd HealthEasy-G
```

```
cp .env.example .env    # then fill in your own values
```

```
npm install              # generates the Prisma client automatically
```

```
npm run db:setup:cloud   # schema + seed on the online database
```

```
npm run db:setup:local   # schema + seed on the offline database
```

```
npm run dev              # http://localhost:3000
```

The system runs against **either a local PostgreSQL (offline) or a managed cloud PostgreSQL (online)** — the same schema and the same seed, selected by DATABASE_TARGET in .env. See .env.example for every required variable.

Checks

```
npm run typecheck        # TypeScript, no emit
```

```
npm test                  # 53 tests
```

```
npm run build             # production build
```

CI runs these three steps against a throwaway PostgreSQL on every push and pull request ([.github/workflows/ci.yml](#)).

6. Security Model

Authentication and authorisation are enforced **on the server**. The browser is never trusted to report who it is or what role it holds.

Layer	File	Responsibility
Session	src/lib/session.ts	HMAC-SHA256 signed token in an httpOnly cookie; forged, re-signed or expired tokens are rejected
Perimeter	src/proxy.ts	Rejects unauthenticated traffic before any page or route handler runs; enforces each role's allowedRoutes
Policy	src/lib/api-policy.ts	Declares who may call each endpoint and method — fails closed for anything unlisted
Enforcement	src/lib/api-guard.ts	withAuth() wraps every route handler and applies the policy
Validation	src/lib/validation.ts	Type, length and range checks on safety-critical input
Rate limiting	src/lib/rate-limit.ts	Throttles repeated failed sign-in attempts

Consequences worth knowing:

- The acting identity on every write — audit entries, vitals, consultation notes, dispensing, payments — is taken from the session, never from the request body, so records cannot be filed under another person's name.
- The **Data Protection Commission constraint on the Super Admin is enforced, not merely documented**: the system administrator cannot read /api/patients, /api/vitals or /api/encounters. Executive dashboards read /api/stats, which returns aggregate counts containing no patient identifiers.

- Changing the role displayed in the browser grants nothing; the server reads the role from the signed cookie.

7. Where the Data Comes From

Every clinical, financial and administrative screen reads live PostgreSQL rows through the API. There is **no fallback sample data in the application** — an empty table produces an empty state, never a fabricated record.

src/lib/adapters.ts is the single translation point between Prisma rows and the client-facing types, so storage-oriented names and SCREAMING_SNAKE enums (IN_CONSULTATION, bedType, totalClaimGhc) become what the interface renders (In Consultation, type, totalClaimAmountGhc).

DHIMS2 figures are **computed live** from encounters, patients, admissions and claims. Only the two figures the system has no other source for — deaths and maternal deliveries — are entered by the records officer and stored in DhimsMonthlyReturn, exactly as on the paper return.

8. Appearance

Light is the default theme across the public site and the staff portal. Users can switch to dark from the toggle in the dashboard header or the public site navigation; the choice is stored under hms_theme and shared by both. A blocking script in src/app/layout.tsx applies the saved theme before first paint, so dark-mode users never see a flash of light content.

9. Known Limitations

These are stated plainly rather than omitted; each is expanded in docs/Maintenance_and_Evolution.md.

1. **No live NHIA or NIA integration.** Ghana Card and NHIS membership numbers are validated for format and internal consistency, not verified against the national registries, which require an accredited production integration agreement.
2. **PACS imaging is metadata-only.** Radiology orders, reports and accession numbers are managed in full; DICOM image storage and a viewer are not implemented.

3. **Rate limiting is process-local.** `src/lib/rate-limit.ts` holds counters in memory, so on a multi-instance deployment each instance counts separately. A shared store (Redis) is required for production-grade throttling.
4. **DHIMS2 export is on-screen only.** Figures are computed correctly but are not yet emitted as a DHIMS2-format file for upload.
5. **The clinical AI assistant requires a reachable Ollama host.** Without one it falls back to a deterministic three-condition GSTG rule engine, which is safe but narrow.
6. **Single audit-trail retention policy.** Audit entries are append-only but are not yet archived to cold storage on a schedule.

10. Notes for Testing

- The landing page is public; **every other page redirects to sign-in**. A redirect to `/auth/login?next=...` is correct behaviour, not a fault.
- Signing in as the administrator and opening a patient screen will show an access-denied page. **This is the DPC rule working**, not a defect — use the clinician account for patient data.
- Different roles legitimately see different navigation and different data. Compare the two supplied accounts to observe role separation.
- The seeded demonstration data is small on purpose so that every figure on screen can be traced back to a specific database row.

11. Documentation

Full documentation is in docs/:

Document	Contents
Project_Documentation.md	The complete 25-section project report
SRS.md	Software Requirements Specification
Design_Documentation.md	Architecture and all design artefacts
Testing_Report.md	Test strategy, executed evidence and defect log
User_Manual.md	Role-by-role operating instructions

Maintenance_and_Evolution.m d	Maintenance strategy and future roadmap
----------------------------------	---

12. Group Members

#	Name	Student ID	Major Contribution
1	Ernest Nketia Asubonterng	22424715	Lead System Architect & AI Integrator — software architecture, security model, database design, Ollama/GSTG clinical assistant
2	Nana Kwabena Asare	22424817	Clinical EMR consultation and ICD-10 diagnosis engine
3	Casper Kosi Asense	22425080	Master Patient Index, Ghana Card and NHIS validation
4	Aubrey Owusu Amoah	22424666	PACS radiology orders and diagnostic workflows
5	Thomas Nii Armah Okai	22425782	Pharmacy FEFO stock control and prescription dispensing
6	Abubakari Zubeiru	22425115	Inpatient bed management and DPC audit-log compliance
7	Frank Tandoh	22425049	Responsive layouts, design system and accessibility

13. Acknowledgements

Third-party frameworks, libraries and services used in this project are acknowledged in docs/Project_Documentation.md 24 (References) and in package.json. Principal dependencies: Next.js and React (Vercel, Meta), Prisma, Tailwind CSS, Radix UI, Tabler Icons, bcryptjs, ApexCharts, Ollama, Neon and Vercel.

Clinical and regulatory reference material: Ghana Standard Treatment Guidelines (Ministry of Health), NHIA G-DRG tariff structure, Ghana Health Service DHIMS2 reporting requirements, HeFRA facility licensing framework, and the Data Protection Act, 2012 (Act 843).